

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

1. Logistic (Sigmoid) Function

```
import numpy as np

def logistic_function(x):
    """
    Computes the logistic (sigmoid) function.
    """
    return 1 / (1 + np.exp(-x))
```

2. Log-Loss Function

```
def log_loss(y_true, y_pred):
    """
    Computes binary log loss.
    """
    y_pred = np.clip(y_pred, 1e-10, 1 - 1e-10)
    loss = -(y_true * np.log(y_pred)) - ((1 - y_true) * np.log(1 - y_pred))
    return loss
```

3. Cost Function (Average Log Loss)

```
def cost_function(y_true, y_pred):
    """
    Computes average log loss.
    """
    assert len(y_true) == len(y_pred)
    n = len(y_true)
    loss_vec = log_loss(y_true, y_pred)
    cost = np.sum(loss_vec) / n
    return cost
```

4. Cost Function for Logistic Regression (Vectorized)

```
def costfunction_logreg(X, y, w, b):
    """
    Computes cost for logistic regression model.
    """
    z = np.dot(X, w) + b
    y_pred = logistic_function(z)
    cost = cost_function(y, y_pred)
    return cost
```

Gradient Computation

```
def compute_gradient(X, y, w, b):
    """
    Computes gradients for logistic regression.
    """
    n, d = X.shape
    z = np.dot(X, w) + b
    y_pred = logistic_function(z)

    error = y_pred - y
    grad_w = np.dot(X.T, error) / n
    grad_b = np.sum(error) / n
```

```
return grad_w, grad_b
```

6. Gradient Descent Algorithm

```
def gradient_descent(X, y, w, b, alpha, n_iter, show_cost=False, show_params=True):
    """
    Batch gradient descent for logistic regression.
    """
    cost_history = []
    params_history = []

    for i in range(n_iter):
        grad_w, grad_b = compute_gradient(X, y, w, b)

        w -= alpha * grad_w
        b -= alpha * grad_b

        cost = costfunction_logreg(X, y, w, b)

        cost_history.append(cost)
        params_history.append((w.copy(), b))

        if show_cost and (i % 100 == 0 or i == n_iter - 1):
            print(f"Iteration {i}: Cost = {cost:.6f}")

        if show_params and (i % 100 == 0 or i == n_iter - 1):
            print(f"Iteration {i}: w = {w}, b = {b:.6f}")

    return w, b, cost_history, params_history
```

7. Prediction Function

```
def prediction(X, w, b, threshold=0.5):
    """
    Predicts binary labels using logistic regression.
    """
    z = np.dot(X, w) + b
    y_prob = logistic_function(z)
    y_pred = (y_prob >= threshold).astype(int)
    return y_pred
```

8. Evaluation Metrics (Confusion Matrix, Precision, Recall, F1)

```
def evaluate_classification(y_true, y_pred):
    """
    Evaluates binary classification results.
    """
    TP = np.sum((y_true == 1) & (y_pred == 1))
    TN = np.sum((y_true == 0) & (y_pred == 0))
    FP = np.sum((y_true == 0) & (y_pred == 1))
    FN = np.sum((y_true == 1) & (y_pred == 0))

    confusion_matrix = np.array([[TN, FP],
                                  [FN, TP]])

    precision = TP / (TP + FP) if (TP + FP) > 0 else 0.0
    recall = TP / (TP + FN) if (TP + FN) > 0 else 0.0
    f1_score = (2 * precision * recall) / (precision + recall) if (precision + recall) > 0 else 0.0

    return confusion_matrix, precision, recall, f1_score
```

9. Training on PIMA Indians Diabetes Dataset

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"
```

```

columns = ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
           'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome']

data = pd.read_csv(url, names=columns)

columns_to_clean = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
data[columns_to_clean] = data[columns_to_clean].replace(0, np.nan)
data.fillna(data.median(), inplace=True)

X = data.drop(columns=['Outcome']).values
y = data['Outcome'].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

10. Model Training

```

w = np.zeros(X_train_scaled.shape[1])
b = 0.0
alpha = 0.1
n_iter = 1000

w, b, cost_history, _ = gradient_descent(
    X_train_scaled, y_train, w, b, alpha, n_iter, show_cost=True
)

```

```

Iteration 0: Cost = 0.676575
Iteration 0: w = [0.00991956 0.02441096 0.00873782 0.01116263 0.01171736 0.01566893
 0.00787722 0.01146834], b = -0.015147
Iteration 100: Cost = 0.465441
Iteration 100: w = [0.2543368 0.81844566 0.06829642 0.13758848 0.12862966 0.44347497
 0.19965437 0.18366522], b = -0.658039
Iteration 200: Cost = 0.455913
Iteration 200: w = [0.31408124 1.01845157 0.01606731 0.09664694 0.04049085 0.55737366
 0.22144573 0.17000793], b = -0.799893
Iteration 300: Cost = 0.453874
Iteration 300: w = [ 0.34549333 1.10845968 -0.01332445 0.06851339 -0.01399901 0.61945585
 0.22883654 0.15984656], b = -0.844027
Iteration 400: Cost = 0.453316
Iteration 400: w = [ 0.36291522 1.15398675 -0.02960862 0.05052658 -0.04309738 0.65526548
 0.23241019 0.15382819], b = -0.861008
Iteration 500: Cost = 0.453148
Iteration 500: w = [ 0.37276668 1.17803251 -0.03871108 0.03920436 -0.05839397 0.67618319
 0.2343152 0.15043614], b = -0.868680
Iteration 600: Cost = 0.453096
Iteration 600: w = [ 0.37837525 1.19100185 -0.0438417 0.03217687 -0.06650733 0.68846901
 0.23535386 0.14858208], b = -0.872530
Iteration 700: Cost = 0.453079
Iteration 700: w = [ 0.38157042 1.19808569 -0.04675606 0.02786712 -0.07086644 0.69570176
 0.2359222 0.14759338], b = -0.874582
Iteration 800: Cost = 0.453074
Iteration 800: w = [ 0.38338753 1.20198902 -0.04842282 0.02524911 -0.07323625 0.69996193
 0.23623295 0.14707803], b = -0.875711
Iteration 900: Cost = 0.453072
Iteration 900: w = [ 0.38441855 1.20415428 -0.0493814 0.02367054 -0.07453722 0.70247012
 0.23640263 0.14681555], b = -0.876344
Iteration 999: Cost = 0.453071
Iteration 999: w = [ 0.38499798 1.20535295 -0.0499309 0.02273151 -0.07525175 0.70393437
 0.23649451 0.14668614], b = -0.876699

```

11. Evaluation

```

y_train_pred = prediction(X_train_scaled, w, b)
y_test_pred = prediction(X_test_scaled, w, b)

train_cost = costfunction_logreg(X_train_scaled, y_train, w, b)
test_cost = costfunction_logreg(X_test_scaled, y_test, w, b)

print(f"Train Cost: {train_cost:.4f}")
print(f"Test Cost: {test_cost:.4f}")

```

```

accuracy = np.mean(y_test_pred == y_test) * 100
print(f"Test Accuracy: {accuracy:.2f}%")

conf_matrix, precision, recall, f1 = evaluate_classification(y_test, y_test_pred)

print("Confusion Matrix:\n", conf_matrix)
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1 Score: {f1:.2f}")

```

```

Train Cost: 0.4531
Test Cost: 0.5146
Test Accuracy: 70.78%
Confusion Matrix:
[[82 18]
 [27 27]]
Precision: 0.60
Recall: 0.50
F1 Score: 0.55

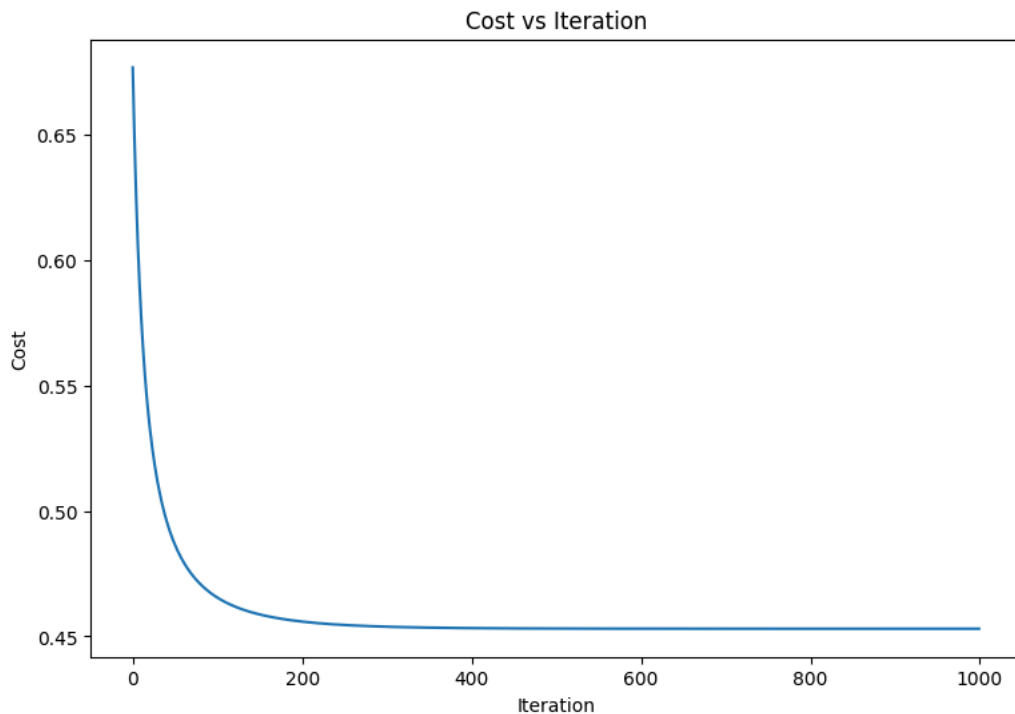
```

12. Cost Convergence Plot

```

plt.figure(figsize=(9,6))
plt.plot(cost_history)
plt.xlabel("Iteration")
plt.ylabel("Cost")
plt.title("Cost vs Iteration")
plt.show()

```



```

# Test model
y_train_pred = prediction(X_train_scaled, w, b)
y_test_pred = prediction(X_test_scaled, w, b)
# Evaluate train and test performance
train_cost = costfunction_logreg(X_train_scaled, y_train, w, b)
test_cost = costfunction_logreg(X_test_scaled, y_test, w, b)
print(f"\nTrain Loss (Cost): {train_cost:.4f}")
print(f"Test Loss (Cost): {test_cost:.4f}")

```

```

Train Loss (Cost): 0.4531
Test Loss (Cost): 0.5146

```

```

# Accuracy on test data

```

```
test_accuracy = np.mean(y_test_pred == y_test) * 100  
print(f"\nTest Accuracy: {test_accuracy:.2f}%")
```

Test Accuracy: 70.78%